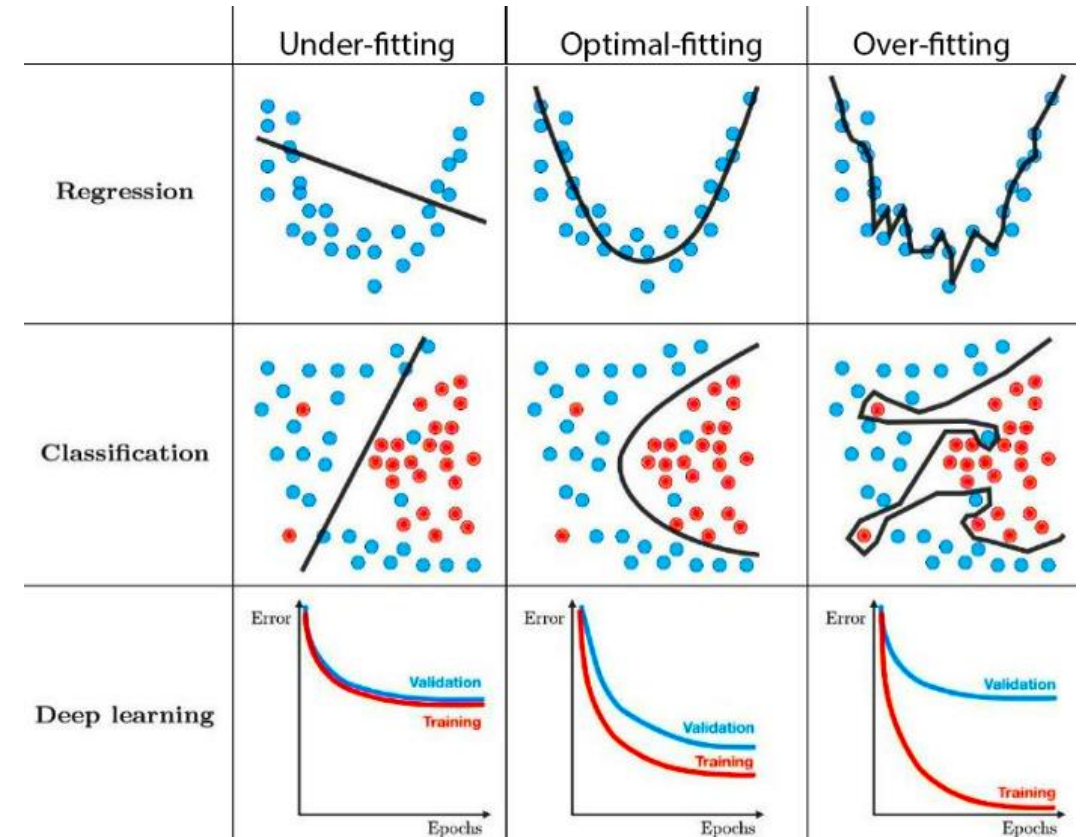


Some Important Concepts

ML and pattern recognition

- The objective of a machine learning model is to learn patterns from training data by minimizing a loss function, enabling it to make accurate predictions or decisions on unseen data.



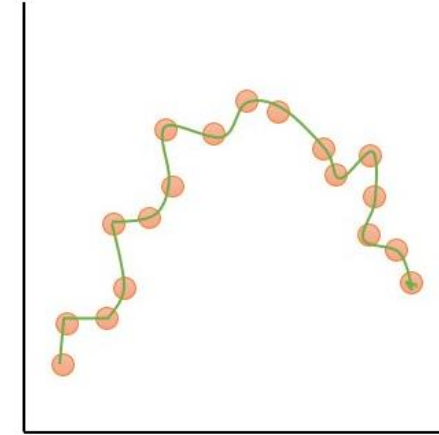
Bias and Variance

- With [larger data sets](#), various implementations, algorithms, and learning requirements, it has become even more complex to create and evaluate ML models since all those factors directly impact the overall accuracy and learning outcome of the model.
- This is further skewed by false assumptions, noise, and outliers.
- Bias and variance are used to analyze the two major sources of prediction error in a machine learning model and to achieve a balance that minimizes the total error and improves generalization.
- [Machine Learning Fundamentals: Bias and Variance - YouTube](#)

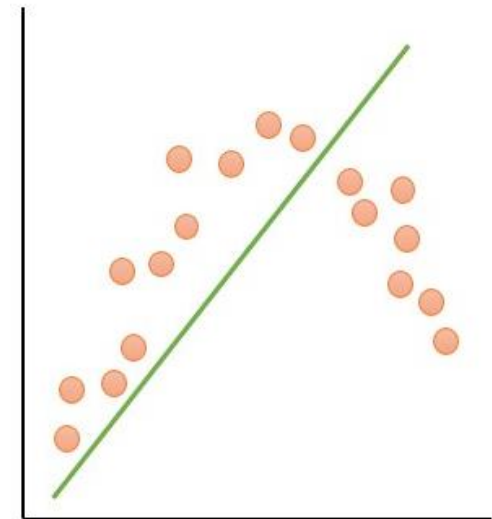
Bias

- **Bias** in machine learning refers to a systematic error introduced by a model due to overly simplistic assumptions made during the learning process.
- It represents the difference between the model's average prediction and the actual (ground truth) value.
- Bias indicates how well a model can learn patterns from the training data.
 - A low bias model will closely match the training data set.
 - A model with a higher bias would not match the data set closely.
 - A **high-bias model** typically:
 - Fails to learn important data patterns
 - Uses overly simple assumptions
 - Results in underfitting
 - Shows poor performance on both training and unseen data
- In summary, **high bias occurs when the model is too simple to capture the complexity of the problem.**

Low Bias, Overfitting

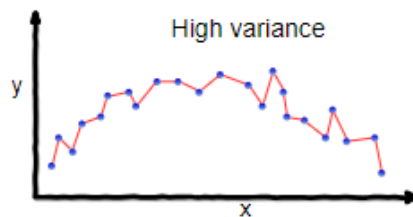


High Bias, Underfitting

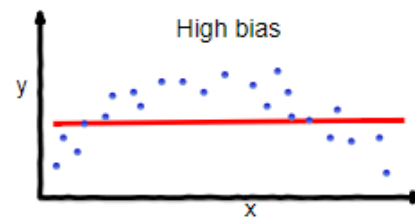


Variance

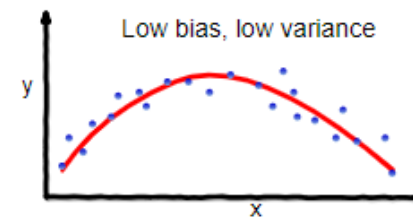
- Variance measures how much a model's predictions change when it is trained on different datasets.
- It measures how much the model's predictions change when it is trained on different subsets of the same dataset.
- A model with **high variance** learns the training data too closely, including noise and minor fluctuations. As a result, it performs well on training data but poorly on unseen data (test data).
- A **high-variance model** typically:
 - Learns unnecessary details and noise from the data
 - Shows overfitting
 - Performs well on training data but poorly on test data
 - Has poor generalization ability
- In summary, **high variance occurs when a model is too complex and overly dependent on the training data.**
- The model will not be generalized



overfitting



underfitting



Good balance

Bias vs variance: A trade-off

- Bias and variance are inversely connected. It is difficult to have an ML model with a low bias and a low variance (ideal case).
- The **bias–variance trade-off** describes the balance between a model's simplicity and its ability to generalize to unseen data.
 - **Bias** occurs when a model is too simple and cannot capture the underlying patterns in the data, leading to underfitting.
 - **Variance** occurs when a model is too complex and becomes highly sensitive to the training data, leading to overfitting.
- As model complexity increases:
 - Bias decreases because the model can learn more patterns.
 - Variance increases because the model becomes more sensitive to noise.
- The goal of machine learning is to find an **optimal balance** where both bias and variance are minimized, resulting in the lowest possible prediction error on new data.
- In summary, **a good model is neither too simple nor too complex**—it achieves a balance between bias and variance.

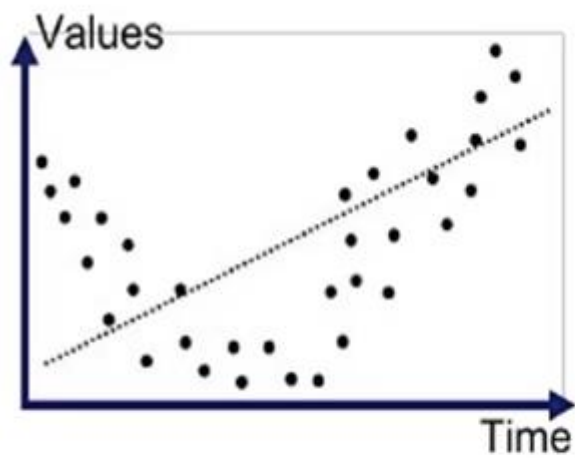
Common algorithms and their expected behavior regarding bias and variance:

Algorithm	Bias	Variance
Linear Regression	High Bias	Less Variance
Decision Tree	Low Bias	High Variance
Bagging	Low Bias	High Variance (Less than Decision Tree)
Random Forest	Low Bias	High Variance (Less than Decision Tree and Bagging)

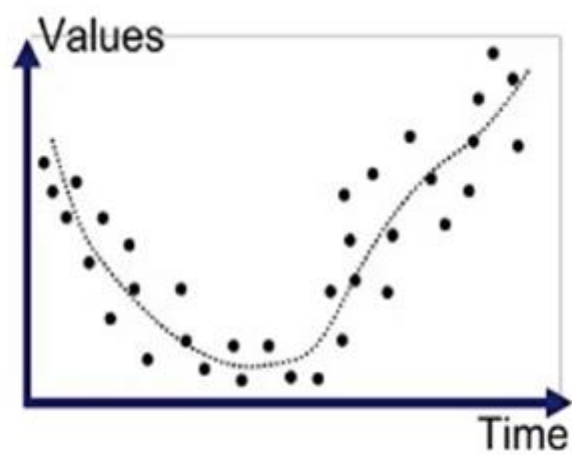
Overfitting

- Overfitting occurs when a machine learning model achieves very high accuracy on the training data but low accuracy on the testing or unseen data because it has memorized the training data instead of learning general patterns.
- Defined as Low bias and high variance
- **Characteristics of Overfitting**
 - Very high training accuracy
 - Low testing/validation accuracy
 - Poor generalization to unseen data
 - Learns noise and outliers instead of underlying patterns

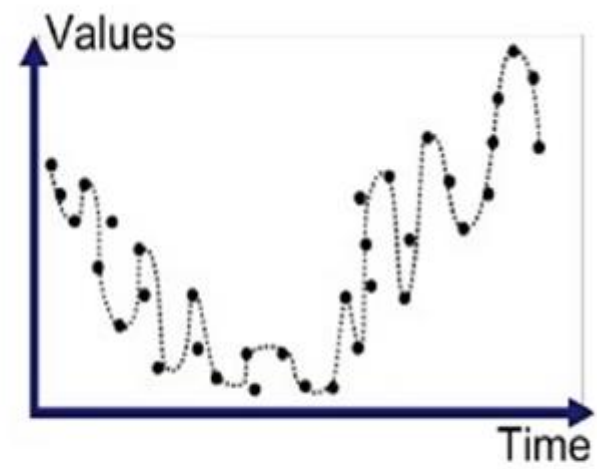
Dataset	Accuracy
Training Data	99%
Testing Data	75%



Underfitted



Good Fit/Robust



Overfitted

How to handle overfitting?

1. Regularization

- Regularization helps to solve over fitting problem by adding a **penalty** term to the **objective function** and control the model complexity using that penalty term.
- For linear or logistic regression, suppose the objective is to minimize the loss (error) function.

$$J(\theta) = \text{Loss}$$

With Regularization, a penalty term is added to the loss:

$$J(\theta) = \text{Loss} + \lambda \times \text{Penalty}$$

where

- **Loss** = prediction error
- **Penalty** = discourages large weights
- λ (**lambda**) = regularization parameter controlling the penalty strength
- Small $\lambda \rightarrow$ little regularization
- Large $\lambda \rightarrow$ strong regularization

How does Regularization Work?

- Example

- Let's consider the simple linear regression equation:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \dots + \beta_n x_n$$

The fitting procedure involves a loss function, such as SSE (RSS)/ MSE.

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

- Regularization shrinks or regularizes these learned estimates (weights) towards zero, by adding the loss function with penalty term

$$\text{Loss} = \text{Error} + \lambda \times \text{Penalty}$$

- **Penalty** → measures model complexity (usually large weights)
 - **λ (lambda)** → controls *how strong* the regularization is

Types of regularization

There are mainly two types of regularization techniques:

- Ridge Regression ([Regularization Part 1: Ridge \(L2\) Regression - YouTube](#))
- Lasso Regression ([Regularization Part 2: Lasso \(L1\) Regression - YouTube](#))

Ridge Regression (L2 regularization)

- In this technique, the cost function is altered by adding the **Ridge Regression penalty** term to it.
- The equation for the cost function in ridge regression will be:

$$\text{LOSS}_{\text{Ridge}} = \underbrace{\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2}_{\text{RSS}} + \underbrace{\lambda \sum_{j=1}^p \beta_j^2}_{\text{Penalty}}$$

- Penalty depends **only on coefficients**
- Intercept β_0 is **NOT** penalized
- $\lambda \geq 0$ controls strength

- Loss function forces the model to fit the data, while regularization controls how large the coefficients are.

- During weight optimization (gradient descent),

Gradient w.r.t. β_j

$$\frac{\partial \text{Loss}}{\partial \beta_j} = -2 \sum_{i=1}^n x_{ij} (y_i - \hat{y}_i) + 2\lambda\beta_j$$

Update rule

$$\beta_j \leftarrow \beta_j + 2\eta \sum_{i=1}^n x_{ij} (y_i - \hat{y}_i) - 2\eta\lambda\beta_j$$

That last term continuously **shrinks coefficients toward zero**. (This is why ridge is also called **weight decay**.)

Shrinking coefficients reduces model sensitivity to noise, stabilizes learning in the presence of correlated features, and improves generalization to unseen data.

- What is the significance of shrinking the weights (coefficients)?

Large coefficients mean the model is **extremely sensitive** to inputs.

Example:

$$\hat{y} = 100x_1 - 98x_2$$

- A tiny change in x_1 or $x_2 \rightarrow$ huge change in prediction
- Noise gets amplified
- Model becomes unstable

Ridge pushes this toward:

$$\hat{y} = 3x_1 - 2.9x_2$$

Hence, predictions stop swinging wildly.

- Thus, the penalty term regularizes the coefficients of the model, and hence **ridge regression reduces the amplitudes of the coefficients** that decreases the complexity of the model.
- As we can see from the above equation, if the values of λ **tend to zero, the equation becomes the cost function of the linear regression model**. Hence, for the minimum value of λ , the model will resemble the linear regression model.
- A general linear or polynomial regression will fail if there is high collinearity between the independent variables, so to solve such problems, Ridge regression can be used.
- It helps to solve the problems if we have more parameters than samples.

Lasso Regression (L1 regularization):

- Lasso regression is another regularization technique to reduce the complexity of the model. It stands for **Least Absolute and Selection Operator**.
- It is similar to the Ridge Regression except that the penalty term contains only the absolute weights instead of a square of weights.

$$\sum_{i=1}^M (y_i - y'_i)^2 = \sum_{i=1}^M \left(y_i - \sum_{j=0}^n \beta_j * x_{ij} \right)^2 + \lambda \sum_{j=0}^n |\beta_j|$$

- Since it takes absolute values, it can shrink the coefficients (weights) to 0, whereas Ridge Regression can only shrink it near to 0.
- Some of the features in this technique are completely neglected for model evaluation.
- Hence, the Lasso regression can help us to reduce the overfitting in the model as well as the feature selection.

What happens during gradient descent?

Gradient of L1 penalty

$$\frac{\partial}{\partial \beta_j} |\beta_j| = \text{sign}(\beta_j) \quad (\text{undefined at } 0)$$

Update rule:

$$\beta_j \leftarrow \beta_j + 2\eta \sum x_{ij}(y_i - \hat{y}_i) - \eta\lambda \text{sign}(\beta_j)$$

The penalty term is:

- **Constant in magnitude**
- Doesn't depend on size of β_j
- At $\beta_j=0$, derivative is undefined $\rightarrow$ **subgradient interval, forcing the coefficients get stuck at zero**
- Hence, Small coefficients get pushed **all the way to zero, leading to automatic feature selection**

Given function

$$f(x) = |-x|$$

First simplify:

$$|-x| = |x|$$

Derivative of $|x|$

$$\frac{d}{dx}|x| = \begin{cases} +1 & x > 0 \\ -1 & x < 0 \\ \text{undefined} & x = 0 \end{cases}$$

So,

$$\frac{d}{dx} |-x| = \frac{d}{dx} |x| = \begin{cases} +1 & x > 0 \\ -1 & x < 0 \\ \text{undefined} & x = 0 \end{cases}$$

- Regularization is generally useful in the following situations:
 - Large number of variables
 - Low ratio of number observations to number of variables (less number of samples, high number of features)
 - High Multi-Collinearity
- In regularization, we reduce the magnitude of the features by keeping the same number of features.
- Hence, it maintains accuracy as well as a generalization of the model.

General Observations

- In general, L2 performs better than L1 regularization.
- L2 is efficient in terms of computation.
- There is one area where L1 is considered as a preferred option over L2. L1 has in-built feature selection for sparse feature spaces. For example, you are predicting whether a person is having a brain tumor using more than 20,000 genetic markers (features). It is known that the vast majority of genes have little or no effect on the presence or severity of most diseases.
- **Ridge regression** is mostly used to reduce the overfitting in the model, and it includes all the features present in the model. It reduces the complexity of the model by shrinking the coefficients.
- **Lasso regression** helps to reduce the overfitting in the model as well as feature selection.

- 2. Early stopping: Early stopping is a form of regularization to avoid overfitting when training a learner with an iterative method, such as gradient descent.
 - While training neural networks, we iteratively use gradients from the training data and try to make the model fit better approximate the underlying real-world function. In a sense, this method allows you to stop at or near the optimal fitting point. Thereby preventing overfitting to the training set and reducing generalization error. To decide when to stop, we can monitor certain metrics such as the loss, test_accuracy, val_accuracy and depending on certain conditions being met stop the training
 - The main idea behind early stopping is to stop training before a model starts to overfit.

Early stopping approaches

There are three main ways early stopping can be achieved.

- **A. Training model on a preset number of epochs**

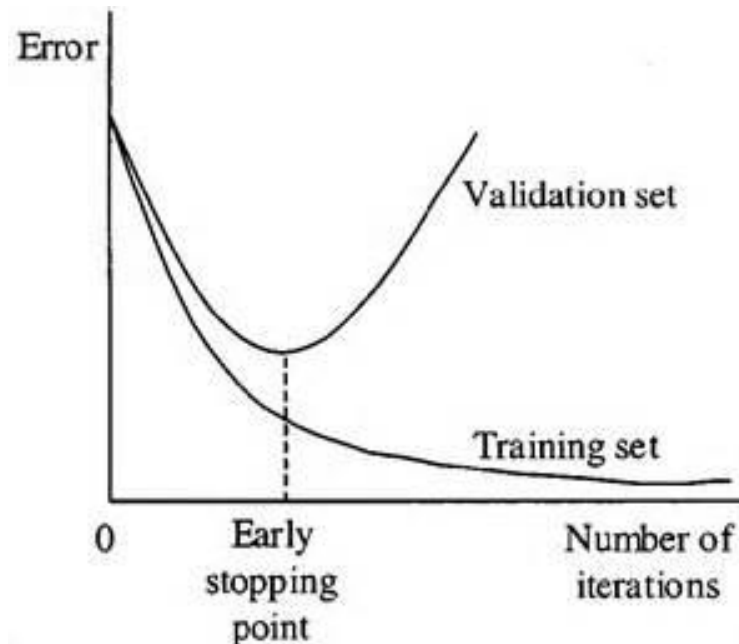
This method is a simple, but naive way to early stop. By running a set number of epochs, we run the risk of not reaching a satisfactory training point. With a higher learning rate, the model might possibly converge with fewer epochs, but this method requires a lot of trial and error. Due to the advancements in machine learning, **this method is obsolete.**

- **b. Stop when the loss function update becomes small**

This approach is built on the fact that the weight updates in gradient descent become significantly smaller as the model approaches minima. Usually, the training is stopped when the update becomes as small as 0.001, as stopping at this point minimizes loss and saves computing power by preventing any unnecessary epochs. However, overfitting might still occur.

- **c. Validation set strategy**

This technique is the most popular early stopping approach. To understand how it works, it's important to look at how training and validation errors change with the number of epochs (as in the figure). The training error decreases exponentially until increasing epochs no longer have such a large effect on the error. The validation error, however, initially decreases with increasing epochs, but after a certain point, it starts increasing. This is the point where a model should be early stopped as beyond this the model will start to overfit.



- Although the validation set strategy is the best in terms of preventing overfitting, it usually takes a large number of epochs before a model begins to overfit, which could cost a lot of computing power.
- A smart way to get the best of both worlds is to **devise a hybrid approach between the validation set strategy and then stop when the loss function update becomes small**. For example, the training could stop when either of them is achieved.

```
# Train the model
```

```
history=model.fit(x_train, y_train, batch_size=128, epochs=10, verbose=1, validation_data=(x_test, y_test))
```

Epoch 1/10

469/469  **47s** 95ms/step - accuracy: 0.8730 - loss: 0.4361 - val_accuracy: 0.9773 - val_loss: 0.0742

Epoch 2/10

469/469  **34s** 73ms/step - accuracy: 0.9788 - loss: 0.0719 - val_accuracy: 0.9818 - val_loss: 0.0574

Epoch 3/10

469/469  **44s** 81ms/step - accuracy: 0.9868 - loss: 0.0449 - val_accuracy: 0.9862 - val_loss: 0.0457

Epoch 4/10

469/469  **31s** 65ms/step - accuracy: 0.9900 - loss: 0.0318 - val_accuracy: 0.9860 - val_loss: 0.0435

Epoch 5/10

469/469  **30s** 64ms/step - accuracy: 0.9938 - loss: 0.0226 - val_accuracy: 0.9849 - val_loss: 0.0435

Epoch 6/10

469/469  **41s** 64ms/step - accuracy: 0.9949 - loss: 0.0172 - val_accuracy: 0.9854 - val_loss: 0.0425

Epoch 7/10

469/469  **41s** 63ms/step - accuracy: 0.9958 - loss: 0.0129 - val_accuracy: 0.9861 - val_loss: 0.0429

Epoch 8/10

469/469  **41s** 63ms/step - accuracy: 0.9976 - loss: 0.0087 - val_accuracy: 0.9866 - val_loss: 0.0434

Epoch 9/10

469/469  **43s** 67ms/step - accuracy: 0.9979 - loss: 0.0071 - val_accuracy: 0.9863 - val_loss: 0.0464

Epoch 10/10

469/469  **40s** 65ms/step - accuracy: 0.9984 - loss: 0.0057 - val_accuracy: 0.9867 - val_loss: 0.0481

- **3. Get more training data:** Although getting more data may not always be feasible, getting more representative data is extremely helpful. Having a larger diverse dataset usually aids in the model performance. You can get a better model that may generalize better. This means that the performance of the model on unseen data (true test set) will be better.
- **4. Augmentation:** If you can't get more data, you can try augmentation to add variation in your data. Augmentation means artificially modifying your existing data by means of transforms that resemble the variation you might expect in the real data.

- **5. Dropout:** The main idea of this technique is to randomly drop units from the neural networks during training.
 - During training, the random dropout samples form a large number of different “thinned” networks. The dropout is achieved by constructing a matrix by drawing from a Bernoulli distribution with probability p (for getting 1) [i.e. the dropout probability is $1-p$] and then doing an element-wise multiplication with the outputs of hidden layers. The following figure shows dropout during the training phase.
- Dropout ensures that no neuron ends up relying too much on other neurons and learns something meaningful instead. Dropout can be applied after convolutional, pooling, or fully-connected layers.
- Since all the neurons aren’t active all the time during the training but with a probability p , the weights need to be scaled with that value during inference time.

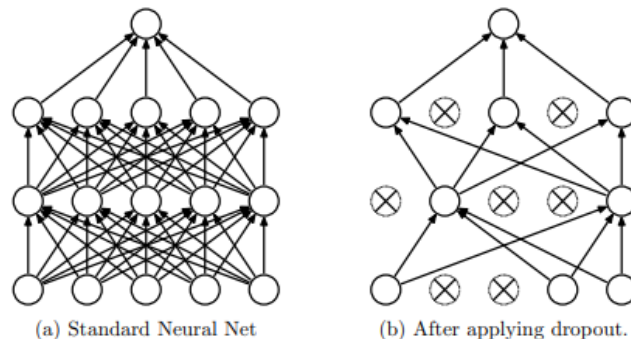
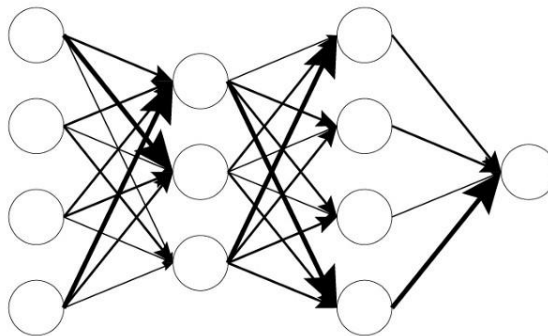


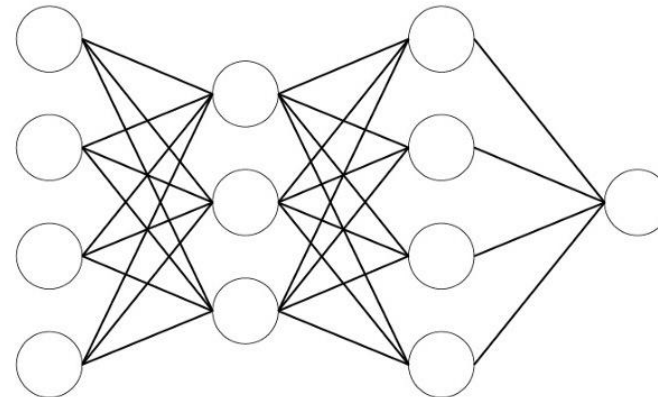
Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

6. Batch Normalization

- It is a process to make neural networks faster and more stable through adding extra layers in a deep neural network.
- The new layer performs the standardizing and normalizing operations on the input of a layer coming from a previous layer.
- During training, the distribution of activations **keeps changing** as weights update.
This causes:
 - unstable gradients
 - slow training
 - sensitivity to learning rate
- BN stabilizes this by **normalizing activations inside the network**.



- Raw signal
- High interdependency between distributions
- Slow and unstable training



- Normalized signal
- Mitigated interdependency between distributions
- Fast and stable training

- Batch Normalization is added as a **separate layer** in the network.
- But it is **not like Dense or Conv** — it doesn't add new features, it **rescales activations**.
- This normalization step is applied right before (or right after) the nonlinear function.

Linear / Conv $\rightarrow$ BatchNorm $\rightarrow$ Activation (ReLU)

Why here?

- BN normalizes the pre-activation values
- ReLU then works on stable inputs

What is the reason behind the term “Batch” in batch normalization?

- A typical neural network is trained using a collected set of input data called batch. Similarly, the normalizing process in batch normalization takes place in batches, not as a single input.
- BN normalizes pre-activation vectors from hidden layers using the first and the second statistical moments (mean and variance) of the current batch.